Context:
Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.
This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.
By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

Dataset: https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb
The data is available in 8 csv files:

1. customers.csv
2. sellers.csv
3. order_items.csv
4. geolocation.csv
5. payments.csv
6. reviews.csv
7. orders.csv
8. products.csv

Problems with solutions:

Problem Statement:
Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

1. **Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**

   A. Data type of all columns in the "customers" table.

Sol:

```
SELECT
column_name,
data_type
 FROM
 target.INFORMATION_SCHEMA.COLUMNS
```

```
WHERE
table_name = 'customers'
```

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Results per page: 50 1 – 5 of 5

Insights:

- Output contains 5 rows and two columns.
- Out of which 4 are of string data type and 1 is integer.

B. Get the time range between which the orders were placed.

Sol:

```
SELECT
min(order_purchase_timestamp) as first_order_time,
max(order_purchase_timestamp) as Last_order_time
from
target.orders
```

Row	first_order_time	Last_order_time
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Results per page: 50 1 – 1 of 1

Insights:

- The time range is between 2016 to 2018 in which total orders were placed.
- Months are in between September 2016 and October 2018.

C. Count the Cities & States of customers who ordered during the given period.

Sol:

```
select
  COUNT(DISTINCT c.customer_city) AS total_cities,
  COUNT(DISTINCT c.customer_state) AS total_states
```

```
from target.customers c
join target.orders o
on c.customer_id = o.customer_id
where o.order_purchase_timestamp between
(select min(order_purchase_timestamp) from target.orders) and (select
max(order_purchase_timestamp) from target.orders)
```

Row	total_cities	total_states
1	4119	27

Results per page: 50 1 – 1 of 1

Insights:

- Clearly we can see there are 4119 cities.
- And 27 states between the given time frame.
- There is only one row.

2. In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

Sol:

```
SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
    COUNT(*) AS total_orders
FROM target.orders
GROUP BY EXTRACT(YEAR FROM order_purchase_timestamp)
ORDER BY order_year;
```

Row	order_year	total_orders
1	2016	329
2	2017	45101
3	2018	54011

Results per page: 50 1 – 3 of 3

Activate Windows

Insights:

- From 2016 to 2017 there is a quantum leap in orders which is more than 100 times.
- And from 2017 to 2018 there is still a significant growth in orders.
- Yes there is a growing trend in the number of orders placed.

B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Sol:

```
SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
    EXTRACT(MONTH FROM order_purchase_timestamp) AS order_Month,
    COUNT(*) AS total_orders
FROM target.orders
GROUP BY EXTRACT(YEAR FROM order_purchase_timestamp),EXTRACT(MONTH FROM
order_purchase_timestamp)
ORDER BY order_year,order_Month;
```

Row	order_year	order_Month	total_orders
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

Results per page: 50 1 – 25 of 25

Insights:

- For 2016 orders are placed for just three months with most orders placed in October.
- For years 2017 and 2018 orders are placed in almost all months with gradual increase in orders.

C. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

Sol:

```
WITH CTE AS (
 SELECT
 CASE
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
'DAWN'
```

```
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
'MORNING'
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
'AFTERNOON'
      ELSE 'NIGHT'
    END AS TIME_SEGMENTATION,
    COUNT(*) AS TT_ORDERS
  FROM target.orders
  WHERE order_purchase_timestamp IS NOT NULL
  GROUP BY TIME_SEGMENTATION
)

SELECT *
FROM CTE
ORDER BY TT_ORDERS DESC;
```

Row	TIME_SEGMENTATION	TT_ORDERS
1	AFTERNOON	38135
2	NIGHT	28331
3	MORNING	27733
4	DAWN	5242

Results per page: 50 1 – 4 of 4

Insights:

- Clearly we can see that most of the orders are placed in the afternoon.
- Followed by night and morning.
- And the least orders are placed at dawn.

3. Evolution of E-commerce orders in the Brazil region:

A.Get the month on month no. of orders placed in each state.

Sol:

```
SELECT c.customer_state,
EXTRACT(YEAR FROM o.order_purchase_timestamp) order_year,
EXTRACT(MONTH FROM o.order_purchase_timestamp) order_month,
COUNT(order_id) as no_of_orders
 FROM
  target.orders o
 JOIN target.customers c
 ON o.customer_id = c.customer_id
 WHERE order_purchase_timestamp IS NOT NULL
 GROUP BY c.customer_state,
```

```
EXTRACT(YEAR FROM o.order_purchase_timestamp),
EXTRACT(MONTH FROM o.order_purchase_timestamp)
ORDER BY c.customer_state,order_year,order_month;
```

Row	customer_state	order_year	order_month	no_of_orders
1	AC	2017	1	2
2	AC	2017	2	3
3	AC	2017	3	2
4	AC	2017	4	5
5	AC	2017	5	8
6	AC	2017	6	4
7	AC	2017	7	5
8	AC	2017	8	4
9	AC	2017	9	5
10	AC	2017	10	6

Results per page: 10 1 – 10 of 565

Activate Windows

Insights:

- Generally there are more than one order every month.
- BA has most of the orders every month compared to other states.

B. How are the customers distributed across all the states?

Sol:

```
WITH CTE_1 AS ((SELECT
 customer_state,COUNT(DISTINCT customer_unique_id) as no_of_customers
 FROM target.customers
 GROUP BY customer_state
 ORDER BY no_of_customers DESC
 ))

 SELECT *,ROUND(100*(no_of_customers/(SELECT COUNT(DISTINCT
 customer_unique_id) FROM target.customers)),2) AS cust_percentage_per_state
        FROM CTE_1
 ORDER BY cust_percentage_per_state DESC;
```

Row	customer_state	no_of_customers	cust_percentage_...
1	SP	40302	41.94
2	RJ	12384	12.89
3	MG	11259	11.72
4	RS	5277	5.49
5	PR	4882	5.08
6	SC	3534	3.68
7	BA	3277	3.41
8	DF	2075	2.16
9	ES	1964	2.04
10	GO	1952	2.03

Results per page: 10 1 – 10 of 27

Activate Windows

Insights:

- From the above output, the state with highest customers is SP with 41.94 of total customers.

- The state with least customers is RR with just 0.05 percent of total customers.
- The top three states are SP,RJ,MG with most of the customers' distribution.
- There are 27 rows of information.

4.Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A.Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
You can use the "payment_value" column in the payments table to get the cost of orders.

Sol:

```
WITH cte_1 AS (
    SELECT
        EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
        EXTRACT(MONTH FROM o.order_purchase_timestamp) AS order_month,
        ROUND(SUM(p.payment_value),2) AS total_payment
    FROM target.orders o
    JOIN target.payments p
        ON o.order_id = p.order_id
    GROUP BY EXTRACT(YEAR FROM o.order_purchase_timestamp),
    EXTRACT(MONTH FROM o.order_purchase_timestamp)
),
cte_2 AS (
    SELECT
        order_year,
        order_month,
        total_payment
    FROM cte_1
    WHERE order_year BETWEEN 2017 AND 2018
      AND order_month BETWEEN 1 AND 8
),
cte_3 as
(SELECT
        order_year,
        ROUND(SUM(total_payment), 2) AS total_payment_till_aug
    FROM cte_2
    GROUP BY order_year),
cte_4 AS (
    SELECT
        *,
```

```
        LAG(total_payment_till_aug) OVER (ORDER BY order_year) AS
prev_year_payment
    FROM cte_3
),
cte_5 AS (
    SELECT
        order_year,
        total_payment_till_aug,
        prev_year_payment,
        ROUND(
            ((total_payment_till_aug - prev_year_payment) /
prev_year_payment) * 100,
            2
        ) AS percent_increase
    FROM cte_4
    WHERE prev_year_payment IS NOT NULL
)
SELECT *
FROM cte_5;
```

Row	order_year	total_payment_til...	prev_year_payment	percent_increase
1	2018	8694733.84	3669022.12	136.98

Results per page: 50 1 – 1 of 1

Activate Windows
Go to Settings to activate Windows.

Insights:

- Percentage increase from year 2017 to 2018 between jan to aug month is 136.98 percent.
- Output contains a single row with order year of 2018 and previous year payment info that is 2017 and payment info till 2018.

B.Calculate the Total & Average value of order price for each state.

Sol:

```
select
c.customer_state,round(sum(payment_value),2) as
total_order_price,
round(round(sum(payment_value),2)/count(*),2) as
avg_value_of_order
from target.customers c
left join target.orders o
on c.customer_id = o.customer_id
left join target.payments p
```

```
on o.order_id = p.order_id
group by c.customer_state
order by total_order_price;
```

Row	customer_state	total_order_price	avg_value_of_order
1	RR	10064.62	218.8
2	AP	16262.8	232.33
3	AC	19680.62	234.29
4	AM	27966.93	181.6
5	RO	60866.2	233.2
6	TO	61485.33	204.27
7	SE	75246.25	208.44
8	AL	96962.06	227.08
9	RN	102718.13	196.78
10	PI	108523.97	207.11

Results per page: 10 1 – 10 of 27

Insights:

- RR has the lowest total order value and SP has the highest order value.
- RR is a low performing state and SP is the high performing state.
- PB has the highest avg value and SP has the lowest avg value.

C. Calculate the Total & Average value of order freight for each state.

Sol:

```
select
c.customer_state,round(sum(i.freight_value),2) as
total_freight_price,
round(round(sum(i.freight_value),2)/count(*),2) as
avg_value_of_freight
from target.customers c
left join target.orders o
on c.customer_id = o.customer_id
left join target.order_items i
on o.order_id = i.order_id
group by c.customer_state
order by total_freight_price;
```

Row	customer_state	total_freight_price	avg_value_of_frei...
1	RR	2235.19	42.98
2	AP	2788.5	34.01
3	AC	3686.75	40.07
4	AM	5478.89	33.01
5	RO	11417.38	40.2
6	TO	11732.68	37.13
7	SE	14111.47	36.18
8	AL	15914.59	35.68
9	RN	18860.1	35.45
10	MS	19144.03	23.2

Results per page: 10 1 – 10 of 27

Insights:

- RR has the lowest freight_value and SP has the highest freight_value.
- RR has the highest average value and SP has the lowest average value
- There are a total 27 rows of information.

5.Analysis based on sales, freight and delivery time.

A.Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:
Sol:

```
select
date_diff(order_delivered_customer_date,order_purchase_timestamp,day)
time_to_deliver,
date_diff(order_delivered_customer_date,order_estimated_delivery_date,
day) diff_estimated_delivery_date
from target.orders
```

Row	time_to_deliver	diff_estimated_d...
1	35	-16
2	30	-28
3	30	12
4	54	36
5	43	-6
6	36	-14
7	29	-20
8	30	-19
9	44	-5
10	68	18

Results per page: 10 1 – 10 of 99441

Activate Windows
Go to Settings to activate Windows.

Insights:

- From the above table we have two columns which are time_to_deliver and diff_estimated_delivery.
- Some values are negative in the diff_estimated_delivery column which indicates that items were delivered much before the estimated delivery date.
- In the time_to_deliver column we can see that minimum delivery time is mostly one month.
- There are a total of 99441 rows of information.

Recommendation:

- Keep the difference between actual delivery time and estimated delivery time very low as much as possible that is not more than 1 week.

B.Find out the top 5 states with the highest & lowest average freight value.

Sol:

```
with cte_1 as (select c.customer_state,round(sum(i.freight_value),2)
total_freight_value,
       round(round(sum(i.freight_value),2)/count(*),2)
        avg_freight_value
        from target.customers c
        left join target.orders o
        on c.customer_id = o.customer_id
        left join target.order_items i
        on o.order_id = i.order_id
        group by c.customer_state
        order by total_freight_value
       ),
       cte_2 as (select *,
                 dense_rank() over (order by avg_freight_value desc)
as top5_high_rank,
                 dense_rank() over (order by avg_freight_value asc
                 )as top5_low_rank
                 from cte_1)
       select customer_state,
         total_freight_value,
         avg_freight_value,
         top5_high_rank,
         top5_low_rank
           from cte_2
       where top5_high_rank <= 5 OR top5_low_rank <= 5
       ORDER BY avg_freight_value DESC;
```

Row	customer_state	total_freight_value	avg_freight_value	top5_high_rank	top5_low_rank
1	RR	2235.19	42.98	1	27
2	PB	25719.73	42.44	2	26
3	RO	11417.38	40.2	3	25
4	AC	3686.75	40.07	4	24
5	PI	21218.2	39.0	5	23
6	DF	50625.5	20.91	23	5
7	RJ	305589.31	20.83	24	4
8	MG	270853.46	20.49	25	3
9	PR	117851.68	20.36	26	2
10	SP	718723.07	15.03	27	1

Results per page: 50 1 – 10 of 10

Activate Windows

Insights:

- Top5_high_rank column indicates those values where rank 1 has the highest average freight value, subsequently rank 2 has the second highest and so on.
- Similarly the top5_low_rank column indicates those values with rank 1 with the lowest average freight value,subsequently rank 2 shows second lowest value and so on.
- RR has the highest average freight value and SP has the lowest average freight value.

C.Find out the top 5 states with the highest & lowest average delivery time.

Sol:

```
with cte_1 as (select c.customer_state,
date_diff(order_delivered_customer_date,order_purchase_timestam
p,day) as delivery_days
from target.customers c
join target.orders o
on c.customer_id = o.customer_id
where o.order_purchase_timestamp is not null
and o.order_delivered_customer_date is not null),
cte_2 as (
  select customer_state,round(avg(delivery_days),2) as
avg_del_days
  from cte_1
  group by customer_state
),
ranking as (
  select *,
  dense_rank() over (order by avg_del_days desc) high_rank,
  dense_rank() over (order by avg_del_days) low_rank
  from cte_2
)
select
  customer_state,
  avg_del_days
from
  ranking
where
  high_rank <= 5 or low_rank <= 5
order by
 avg_del_days desc;
```

Row	customer_state	avg_del_days
1	RR	28.98
2	AP	26.73
3	AM	25.99
4	AL	24.04
5	PA	23.32
6	SC	14.48
7	DF	12.51
8	MG	11.54
9	PR	11.53
10	SP	8.3

Results per page: 50 1 – 10 of 10

Activate Windows

Insights:

- We have two columns here one is customer state and other is average delivery days.
- Average delivery columns indicate top 5 average delivery time with highest value from 1 to 5 and rows 6 to 10 shows top 5 average delivery time with lowest value.

D.Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.
Sol:

```
with delivery_dates as (select c.customer_state,
date_diff(o.order_delivered_customer_date,o.order_purchase_timestamp,d
ay) as real_delivery_days,
date_diff(o.order_estimated_delivery_date,o.order_purchase_timestamp,d
ay) as estimated_delivery_days
from target.customers c
join target.orders o
on c.customer_id = o.customer_id
where o.order_purchase_timestamp is not null
and o.order_delivered_customer_date is not null
and o.order_estimated_delivery_date is not null),
avg_delivery_dates as (
  select customer_state,round(avg(real_delivery_days),2)
avg_real_del_days,round(avg(estimated_delivery_days),2) as
avg_est_delivery_days,
  round(avg(estimated_delivery_days) - avg(real_delivery_days), 2) as
avg_days_faster
  from delivery_dates
  group by customer_state
)
select * from
avg_delivery_dates
```

```
order by avg_days_faster desc

limit 5;
```

Row	customer_state	avg_real_del_days	avg_est_delivery_...	avg_days_faster
1	AC	20.64	40.72	20.09
2	RO	18.91	38.39	19.47
3	AP	26.73	45.87	19.13
4	AM	25.99	44.92	18.94
5	RR	28.98	45.63	16.66

Insights:

- AC is the state where actual delivery of order is fastest.
- Delivery days are at least 16 days faster than estimated delivery dates in the top 5 states.

6.Analysis based on the payments:

A.Find the month on month no. of orders placed using different payment types.

Sol:

```
select p.payment_type,
extract(month from o.order_purchase_timestamp) order_month,
count(*) no_of_orders
 from target.payments p
 join target.orders o
 on p.order_id = o.order_id
 group by extract(month from order_purchase_timestamp),
 p.payment_type
 order by order_month,no_of_orders
```

Row	payment_type	order_month	no_of_orders
1	debit_card	1	118
2	voucher	1	477
3	UPI	1	1715
4	credit_card	1	6103
5	debit_card	2	82
6	voucher	2	424
7	UPI	2	1723
8	credit_card	2	6609
9	debit_card	3	109
10	voucher	3	591

Results per page: 10 1 – 10 of 50

Activate Windows

Insights:

- There are a total 50 rows in output.
- Highest orders (8350) are placed in May and payment type is credit card.
- Lowest order (1) is placed in September and payment type is not_defined.
- Strong customers trust on credit_card payment methods.
- UPI is the second most used.

B.Find the no. of orders placed on the basis of the payment installments that have been paid.

Sol:

```
select
    payment_installments,
    count(distinct order_id) as total_paid_orders
from target.payments
where payment_value > 0
group by payment_installments
order by payment_installments;
```

Row	payment_installm...	total_paid_orders
1	0	2
2	1	49057
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

Results per page: 10 1 – 10 of 24

Activate Windows

Insights:

- Most customers pay in a single installment.
- The second most preferred instalments are 2 and 3.
- After 7th installments there is a sharp decline.
- Only one customer prefers to pay in 22 and 23 installments.